From Shor Breaking secp256k1 to the Point-Add Challenge

Principles, Scoring Motivation, and Implementation Mapping—A Background Overview

BenHuang2025 @ Brevis

2026-07-05

Abstract

This document lays out both the attack principle and the challenge background as groundwork for further writing, in two parts. **Part I (General Principles)** explains ***how* Shor's algorithm step by step breaks the elliptic-curve discrete logarithm (ECDLP) on secp256k1**: from the problem reduction, the algorithm's steps, and the circuit framework, to the step-by-step evolution of the quantum state and the recovery of the private key after measurement. **Part II (Implementation)** maps this general theory onto the actual secp256k1 point-add challenge[1]: which part of it the challenge actually implements, what scoring and verification rules it obeys, why almost the entire cost of the attack concentrates on the **elliptic-curve point addition** and the optimization focus in turn falls *chiefly* on the **modular inverse (GCD)** inside it (with the surrounding field arithmetic a secondary but still live source of gains), and which file and function each section lands in. To understand "why/how Shor breaks ECDLP," start with **Part I** (§1–§7); to quickly align on "what this challenge is optimizing and what constraints it has," read **Part II** (§8–§11) directly. The two parts are mutually independent and can be used as needed.

Part I
General Principles: How Shor Breaks ECDLP

This part focuses on one thing—**how Shor's algorithm turns the "classically hard problem" of recovering the private key into a computation a quantum machine can carry out efficiently**. It is an as-self-contained-as-possible background overview: after reading it you should be able to *derive* how the private key is computed step by step, understand each step's quantum state and the origin of each constructive/destructive interference, and know the exact position within the whole pipeline of the point-add primitive this challenge optimizes. As for *the concrete optimization of that circuit*, that belongs to the engineering pipeline and is not covered here.

Notation used throughout. $E/\mathbb{F}_p$ is the secp256k1 curve, p its 256-bit prime field characteristic, $\mathcal{O}$ the point at infinity (group identity); P is the base point, of prime order r, so the cyclic subgroup $\langle P\rangle \cong \mathbb{Z}_r$; the public key is $Q = kP$, the private key $k \in \{1, \ldots, r-1\}$ is the attack target; $n = 256$ is the scalar bit width. The two quantum input registers hold t bits each (i.e. t qubits each),

[1] Challenge site: https://www.ecdsa.fail/.

$M = 2^t \gtrsim r$ is the quantum Fourier transform (QFT) modulus, $\omega_M = e^{2\pi i/M}$, $\omega_r = e^{2\pi i/r}$. $\mathbb{Z}_N = \{0, \ldots, N-1\}$ denotes the residues mod N. (Reminder: **the public key is written Q, the QFT modulus is written M**, so the two no longer share a single letter—this is the only departure from many textbooks' notation.)

1 Challenge Overview

1.1 secp256k1 and its elliptic-curve group

secp256k1 is the elliptic curve used by Bitcoin, Ethereum, and a great deal of TLS/signature machinery, defined over the prime field $\mathbb{F}_p$:

$$E: \; y^2 = x^3 + 7 \pmod{p}, \qquad p = 2^{256} - 2^{32} - 977.$$

The affine points (x, y) on the curve, together with a point at infinity $\mathcal{O}$, form an **abelian group** under the "chord-and-tangent" addition law, with $\mathcal{O}$ as identity. For two points $P_1 = (x_1, y_1)$, $P_2 = (x_2, y_2)$ (with $P_1 \neq \pm P_2$), their sum $P_3 = (x_3, y_3) = P_1 + P_2$ is given by

$$\lambda = \frac{y_2 - y_1}{x_2 - x_1}, \qquad x_3 = \lambda^2 - x_1 - x_2, \qquad y_3 = \lambda(x_1 - x_3) - y_1 \tag{1}$$

(for point doubling $P_1 = P_2$ use $\lambda = \frac{3x_1^2}{2y_1}$ instead, since secp256k1's a-coefficient is 0). All arithmetic here is in $\mathbb{F}_p$; **note that (1) contains one division** $(x_2 - x_1)^{-1}$—i.e. one *modular inverse*, which will be the most expensive part of the point-add circuit later (§4.3).

The order of the base point P is a 256-bit prime r (secp256k1's cofactor is 1, so the whole curve group is $\langle P \rangle \cong \mathbb{Z}_r$). **Scalar multiplication** $k \mapsto kP$ is adding P to itself k times; it is the basis of all ECC operations.

1.2 The discrete logarithm problem (ECDLP) and its classical hardness

The security of elliptic-curve cryptography rests on the **elliptic-curve discrete logarithm problem (ECDLP)**:

given the base point P and the public key $Q = kP$, find the scalar k.

The forward direction $k \mapsto kP$ takes only $\mathcal{O}(n)$ point additions via double-and-add, at little cost; but the reverse, recovering k from Q, takes even the best *classical* algorithms (Pollard ρ / baby-step giant-step) $\Theta(\sqrt{r}) \approx 2^{128}$ group operations—utterly infeasible when $r \approx 2^{256}$. This is precisely the foundation of ECDSA's unforgeability: forging a signature is equivalent to solving this ECDLP.

The significance of Shor's algorithm is that a sufficiently large *fault-tolerant* quantum computer can solve for k in **polynomial time** $\mathrm{poly}(n)$ in the scalar bit width, breaking through the 2^{128} classical wall. The following sections lay out the principles of this quantum algorithm.

1.3 The primitive this challenge optimizes, and the roadmap of this part

The circuit ecdsa.fail asks you to optimize is exactly the **innermost primitive** used when attacking ECDLP with Shor's algorithm: on a quantum superposition, **controllably and reversibly** adding a *classical* point to a curve point. It is this one because the entire attack can be summarized in a single sentence:

the total quantum arithmetic of the attack $\approx$ *one elliptic-curve point addition* $\times \mathcal{O}(n)$ *calls.*

This part then unfolds that sentence layer by layer following Shor's flow: first reduce ECDLP to "find a hidden structure" (§2), list the algorithm's steps (§3), draw the circuit framework and locate the primitive (§4), then derive the quantum evolution and interference state by state (§5), and finally explain how, after measurement, k is recovered and *confirmed* (§6).

2 How the problem moves over to the quantum side

This section involves no abstract framework; it settles just one concrete thing—how the number-theoretic problem "given P, Q on the curve, find the k with $Q = kP$" gets step by step **encoded into the preparation and measurement of a quantum state**. The whole conversion is the following four steps.

2.1 Step one: choose a probe function that "hides k in a period"

What we know is a set of *classical* data: the curve parameters (p, base point P, order r) and a public-key point Q; the unknown is k. Classically, this is a search in a space of size $r \approx 2^{256}$, infeasible. The first step of the conversion is to **not solve for k directly, but construct a "probe function"**:

$$f(a,b) \;=\; aP + bQ \;=\; aP + b(kP) \;=\; (a+kb)\,P, \qquad a,b \in \mathbb{Z}. \tag{2}$$

Along the way we use $Q = kP$—P, Q are known constants, and a, b are freely chosen inputs. The key is the **symmetry** of f: shifting the input along the vector $(-k, 1)$ leaves the value of f *unchanged*,

$$f(a-k,\ b+1) = \big((a-k)+k(b+1)\big)P = (a+kb)\,P = f(a,b), \tag{3}$$

and more generally $f(a,b) = f(a',b') \iff (a-a') + k(b-b') \equiv 0 \pmod r$. In other words, **the private key k has been encoded as the "period direction / slope" $(-k, 1)$ of f**—finding this period direction is the same as reading out k. At this point, "compute the discrete log" has been rewritten as "find the period of a function." (Readers familiar with the abstract theory will recognize this as a two-dimensional instance of the abelian hidden subgroup problem; but the four steps below are all we need, with no need to unpack it.)

2.2 Step two: pack a, b and the curve point into qubits

To "find the period" on a quantum machine, first encode all the objects in (2) into qubits:

- **Two input registers** (call them the a register and the b register): t qubits each ($t \approx n = 256$; we write "$\approx$" because r is not a power of 2, we only need $2^t \gtrsim r$, and the exact value has a few bits of slack—see §3 and §6.1), holding the binary representation of scalars a, b respectively.
- **One point register**: a curve point is represented by its two $\mathbb{F}_p$ coordinates (x, y), each a 256-bit field element, so about 2×256 data qubits (plus a handful of ancilla for carries / the coordinate system).

Here two concepts must be kept apart: a **register** is a *group of physical qubits* (hardware, a "container"); whereas the ket $|a\rangle$ denotes the a register being in the *computational basis state* that "encodes the integer a" (it has 2^t basis states, one per $a \in \{0, \ldots, 2^t - 1\}$), and $|(x,y)\rangle$ likewise means the point register encodes the point (x, y). So a computational basis state $|a\rangle|b\rangle|(x,y)\rangle$ is nothing but three bit strings—but once the algorithm runs, these registers are usually in a *superposition* of

basis states (e.g. the a register in $\frac{1}{\sqrt{2^t}}\sum_a|a\rangle$), **no longer a single** $|a\rangle$ (the rigorous state evolution is in §5). The classical probe function f is then realized as a **reversible circuit** U_f acting on these qubits: $|a,b\rangle|\mathcal{O}\rangle \mapsto |a,b\rangle|aP+bQ\rangle$ (its construction is in §4).

2.3 Step three: turn "search for k" into "prepare a superposition carrying the k-period"

With U_f in hand, the core step of the conversion is: **rather than trying** (a,b) **one by one, first prepare all** (a,b) **into a *superposition*, then evaluate** f **on the whole superposition at once**.

1. Apply one layer of H to each input register (each qubit becomes $\frac{1}{\sqrt{2}}(|0\rangle+|1\rangle)$). This H step, *in one shot*, puts both registers *simultaneously* into a uniform superposition over all M^2 basis states $|a,b\rangle$ ($M=2^t$):
$$\frac{1}{M}\sum_{a,b=0}^{M-1}|a,b\rangle\,|\mathcal{O}\rangle.$$

2. Apply U_f *once*. Because quantum evolution is *linear*, U_f acts on *every term* of the sum—so a single application evaluates f on all M^2 inputs at once:
$$\frac{1}{M}\sum_{a,b=0}^{M-1}|a,b\rangle\,|\mathcal{O}\rangle \xrightarrow{U_f} \frac{1}{M}\sum_{a,b=0}^{M-1}|a,b\rangle\,|aP+bQ\rangle.$$

This step is the very essence of quantum computing ("superposition" ≠ "enumeration"). This deserves a close look, especially to clarify that the sum $\sum_{a,b}$ above is *not* a classical "iteration." Here a,b are each t bits, taking values only in $\{0,\dots,M-1\}$ ($M=2^t$, *finite*, not the unbounded integers), M^2 pairs in all. For a classical machine to actually compute f at all these M^2 pairs (a,b), it would have to evaluate it $M^2=2^{512}$ times *one by one*—completely impossible. The quantum effect *looks* the same (the values of f at all (a,b) are already obtained), **yet there is no one-by-one evaluation loop**:

- **Superposition** encodes exponentially many branches into *linearly* many qubits: one register of $t=256$ qubits, each passed through a single H (t gates in all), already generates a superposition of 2^{256} $|a\rangle$'s ($H^{\otimes t}|0\rangle^{\otimes t}=\frac{1}{\sqrt{M}}\sum_a|a\rangle$); the two registers together use $2t$ qubits and $2t$ gates for all M^2 pairs. These M^2 pairs (a,b) are not M^2 independent objects but amplitudes of *one and the same* state vector.

- **Quantum parallelism** makes *one* application of U_f (by linearity) act on all branches at once—one application encodes this *entire lookup table* of "input $\mapsto aP+bQ$" into the state.

So the qubit count and gate count throughout are poly(t), **with no** 2^t **factor**: some t-order qubits carry the computation of 2^{512} branches. The exponential "width" exists only in the state vector and corresponds to no per-item operation—*this is the fundamental difference of quantum from classical.*

But it must also be stressed: this is not a "free brute-force search." This table *cannot be read out*—one measurement collapses to a single random (a,b), and all other branches are lost. So quantum parallelism *by itself brings no speedup.* The real power comes next, from the **interference** of the QFT, which extracts from this "already-computed but not directly readable" table, *in one shot*, a single *global* property—the hidden period of f (that is, the private key k; how the QFT

singles it out is in step four and §5). **Superposition + parallelism + interference** together are the real source of Shor's speedup over classical; parallel preparation alone is not enough.

The result is an **entangled state**: the input (a, b) is correlated with the point value $aP + bQ$, and by (3) the whole state has $(-k, 1)$ as its *hidden period*. An essential shift has happened here—classically solving for k takes about 2^{128} evaluations, whereas the quantum machine does *one* coherent evaluation and thereby completes the computation of f on exponentially many inputs, and the periodicity of k **now physically exists in the structure of this state** (the rigorous ket-by-ket evolution is in §5). The only expensive part is precisely this step's U_f ($\approx 2n$ point additions), i.e. the primitive this challenge optimizes.

2.4 Step four: use the QFT to "read out" the period

The hidden period already exists in the state, but *direct* measurement only collapses randomly and cannot read it out. The last step is to first apply a QFT to each input register, letting **quantum interference** turn the periodic structure into measurable peaks in the frequency domain, then measure the *same pair* of registers. QFT_M is the discrete Fourier transform over $\mathbb{Z}_M$; it maps the a register to its *Fourier conjugate variable* c and b to d—so c, d live on the **same pair of registers** as a, b, just switched from the "position basis" to the "frequency basis," and their values are not equal (just as the classical discrete Fourier transform sends a sequence $\{x_n\}$ to its spectrum $\{\hat{x}_k\}$: here c, d play the role of $\hat{x}_k$). The result is that the measured integer pair (c, d) satisfies, with high probability,

$$d \equiv k\,c \pmod{r}, \tag{4}$$

so as long as $c \not\equiv 0$, $k \equiv d\,c^{-1} \pmod{r}$ follows in one step. At this point a classical number-theoretic problem has been fully converted into a quantum procedure: *encode k as the period of f → pack f into qubits → use one reversible evaluation to pour the period into the superposition → read it out with the QFT*. As for why the QFT can single out (4) and why the peaks fall exactly on this line, that is the interference derivation of §5; how to recover and *confirm* k from (c, d) is §6.

3 Execution Steps

This section gives the panorama first. The whole algorithm is the following **five steps** (hereafter written **Step 1** through **Step 5**; when later sections say "Step 2" they mean Step 2 here); this section states the operator and register scale used by each step, leaving the mathematical unfolding to §5–§6.

Step 1. Prepare the superposition. The two t-bit input registers (the a register, the b register) each go through $H^{\otimes t}$, placing them in a uniform superposition over $\{0, \ldots, M-1\}$ ($M = 2^t$); the point register is initialized to $|\mathcal{O}\rangle$.

Step 2. Reversible evaluation of f (this step is where all the cost lives). Apply the unitary $U_f : |a, b\rangle|\mathcal{O}\rangle \mapsto |a, b\rangle|aP + bQ\rangle$—*internally it is two scalar multiplications, about $2n$ controlled elliptic-curve point additions* (unfolded, with the primitive's location, in §4). It **almost monopolizes** the whole machine's Toffoli and qubit budget, and is the "Step 2" referred to repeatedly below.

Step 3. Double QFT. Apply QFT_M to *each* of the a, b input registers (relatively cheap, see §4.4).

Step 4. Measure. Measure the two input registers, obtaining an integer pair $(c, d) \in \{0, \ldots, M-1\}^2$. (The point register may or may not be measured; it does not affect the result.)

Step 5. Classical recovery + verification. From (c, d), via rounding/modular division, solve for a candidate k, and verify with $kP \stackrel{?}{=} Q$; on failure go back to Step 1 and resample (see §6).

The trade-off in register size. Ideally the two input registers would work directly over $\mathbb{Z}_r$ ($M = r$), making the interference "clean" (§5.3); but real hardware register widths are powers of 2, so take $M = 2^t$ with $2^t \gtrsim r$ (usually $t \approx n$, and to raise the single-shot success rate one can go up to $\approx 2n$). M not being an integer multiple of r brings "leakage," handled by the nearest-integer rounding of §5.4 and §6. The whole algorithm's **cost balance** is: Step 2's $\Theta(n)$ point additions, each containing several modular multiplications and *one modular inverse*, dominate; Steps 1 and 3 use $\mathcal{O}(t)$ and $\mathcal{O}(t^2)$ cheap gates respectively and are negligible. This is exactly why "optimizing the point-add primitive" deserves serious attention.

4 Circuit Framework

4.1 Registers and top-level structure

Figure 1 gives the top-level circuit—only the main structure, not down to the Toffoli level. The four registers from top to bottom are: two t-bit input registers a, b; a **point register** initialized to $|\mathcal{O}\rangle$ (in the circuit, a curve point is represented by its two $\mathbb{F}_p$ coordinates, so about 2×256 data qubits, plus a few for carries/coordinate system); and a bundle of scratch/ancilla that must be cleared after use (*ancilla / scratch qubits*; dashed in the figure, annotated as uncomputed back to $|0\rangle$, reasoning in §4.3). The whole U_f (the reversible evaluation of Step 2) spans these wires.

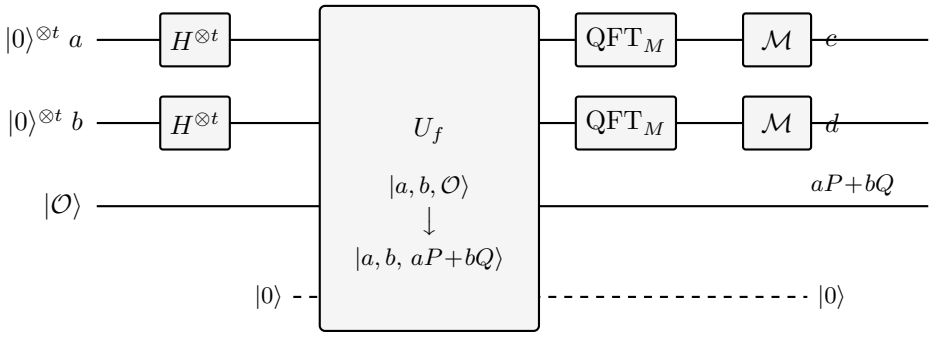

Figure 1: Top-level circuit framework. $H^{\otimes t}$ prepares the superposition; U_f reversibly writes $aP+bQ$ into the point register (scratch cleared after use); after the double QFT_M, measuring the input registers gives (c, d). The internal expansion of U_f is in Figure 2.

4.2 Inside U_f: scalar mult → double-and-add → controlled point-add

The aP, bQ that U_f must compute are both **scalar multiplications**. Expanding the scalar in binary $a = \sum_{i=0}^{t-1} a_i 2^i$ gives

$$aP = \sum_{i=0}^{t-1} a_i \, (2^i P), \qquad bQ = \sum_{j=0}^{t-1} b_j \, (2^j Q), \tag{5}$$

where $2^i P$, $2^j Q$ are all **classical points** (a, b are quantum, but P, Q are known constants, so these multiples can be precomputed classically). So double-and-add unfolds each scalar multiplication

into a string of *controlled* point additions: starting from the accumulator $|\mathcal{O}\rangle$, at step i add the constant point 2^iP to the accumulating point when the control bit $a_i = 1$, and skip when $a_i = 0$. The whole U_f is a cascade of about $2n$ such controlled additions (Figure 2). Each one of them,

"controllably adding a classical point to the accumulating point on a superposition" — **is the primitive this challenge optimizes.**

It is the innermost loop, called $\mathcal{O}(n)$ times, and its internal algebra (next subsection) is where all the cost lives.

What "adding a classical point" concretely is. There is *no* "quantum state times a classical number" operation here—it is exactly the elliptic-curve *point addition* (1) of §1. The accumulator register holds a curve point $R = (x_R, y_R)$ (quantum data, possibly in superposition); to add the constant point $C = 2^iP = (x_C, y_C)$ (a known point already computed classically) and get $R + C$, the circuit follows (1) using the *constants* x_C, y_C and the *quantum* coordinates x_R, y_R in a string of reversible arithmetic (constant subtraction, modular inverse, modular multiplication, …), overwriting R's coordinates *in place* with those of $R + C$—**one operand a constant, the other quantum data**. The control shows up as: the addition happens only when $a_i = 1$. As for the "quantum scalar × classical point" aP, it too is *not* a single multiplication but exactly the accumulation of this string of controlled point additions in (5) (whenever $a_i = 1$, add one 2^iP)—so the innermost layer is always just *point addition*, no other multiplication.

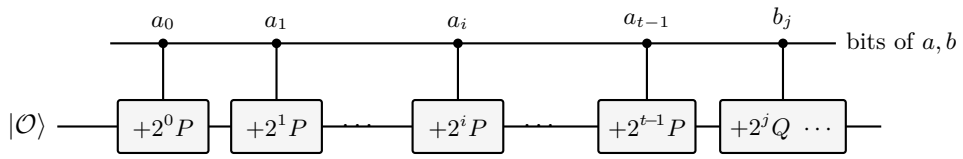

Figure 2: Inside U_f (schematic). The scalar multiplications aP, bQ unfold via double-and-add into about $2n$ controlled point additions. **Every box is the *same* point-add primitive**—when its control bit is 1, it controllably adds a *classical* point to the accumulating point; the boxes differ only in "which classical point is being added" $(2^0P, 2^1P, \ldots, 2^jQ)$, with identical circuit structure (the i in $+2^iP$ stands for any bit position).

4.3 Inside point addition: why the modular inverse is the most expensive, and why it is computed twice

Return to the addition law (1) of §1: one point addition needs several $\mathbb{F}_p$ additions / subtractions / multiplications / squarings, plus **one division** $\lambda = (y_2 - y_1)(x_2 - x_1)^{-1}$. The former are structurally fixed branch-free circuits, whereas that $(\cdot)^{-1}$ is a **modular inverse** $u \mapsto u^{-1} \bmod p$, the most expensive and hardest-to-implement part of the whole circuit. This *"expensive"* has a definite meaning—it contributes the most **Toffoli gates** (and also occupies the most qubits), and the Toffoli count and qubit count are exactly the two factors of the §9 score score = Toffoli × qubit. It is expensive for two reasons:

(a) The modular inverse is inherently data-dependent, yet must be built as an "input-oblivious" fixed-shape circuit. The classical algorithms for modular inverse (extended Euclid / binary GCD / Kaliski) are all **data-dependent**: the iteration count and branch decisions depend on the specific value u. But a quantum circuit is a *fixed unitary*—**the circuit structure cannot**

depend on the (superposed) data (no runtime `if`), and mid-circuit measurement to decide branches would collapse the superposition. So one can only rewrite an inherently "variable-length, branching" algorithm into an **oblivious (input-oblivious), fixed-length, fixed-shape** reversible circuit: unfold all iterations to the worst case ($\Theta(n)$ rounds), replace `if` branches with *coherent controlled swaps*, and use a fixed schedule covering all possible inputs. This makes the modular inverse's gate count far exceed that of the other modular operations, making it the cost center of the point-add primitive.

(b) Reversibility demands "uncomputing" the intermediate garbage bits, nearly doubling the cost. A quantum circuit is unitary evolution and cannot overwrite or discard intermediate results; the large amount of scratch/ancilla produced by the modular-inverse iteration, if left untreated, would **entangle** with the working registers and destroy the interference Shor relies on (§5). So after each inversion, one must *run the inverse of the inversion again* to cleanly uncompute the scratch back to $|0\rangle$—this is the sole reason a *second* Kaliski pass runs in the circuit: besides computing the result, one must also restore it bit-for-bit in reverse, nearly doubling the cost of point addition. (How to optimize these two parts is an engineering matter, not unpacked here.)

4.4 The QFT circuit (why it is relatively cheap)

The QFT_M ($M = 2^t$) of Step 3 has a standard decomposition:

$$\mathrm{QFT}_M : \ |x\rangle \longmapsto \frac{1}{\sqrt{M}} \sum_{y=0}^{M-1} \omega_M^{xy} \, |y\rangle,$$

which on the circuit is one Hadamard per wire, together with $\binom{t}{2}$ controlled phase gates $R_m = \mathrm{diag}(1, e^{2\pi i/2^m})$, for a total of $\mathcal{O}(t^2)$ gates; dropping the exponentially small rotations (approximate QFT) lowers this to $\mathcal{O}(t \log t)$. The key point is: these are all *single/two-qubit* Clifford+phase gates, and compared with Step 2's $\Theta(n)$ point additions, each with many Toffolis, they are **negligible**. This also explains why the whole challenge focuses only on the point-add primitive: the QFT is cheap, and what is expensive is "the reversible evaluation of f."

4.5 How many qubits this circuit needs

Qubit count (circuit width) is one of the most critical parameters of a quantum circuit, and one factor of this challenge's score = (average executed Toffoli count) × (peak qubit count), so it is worth estimating. Counting block by block per the structure above (in units of the scalar bit width $n = 256$):

- **Two input registers**: $t \approx n$ qubits each, $\approx 2n$ total;
- **Point accumulator**: the two $\mathbb{F}_p$ coordinates of one curve point, $\approx 2n$ total (the *classical* point being added is a constant and takes no qubits);
- **Modular-inverse / modular-arithmetic scratch**: the modular inverse (Kaliski) must maintain several n-bit working registers (u, v, r, s and the like) simultaneously, the bulk of the width, several n in all; plus carries, flags, etc. at $\mathcal{O}(\log n)$.

The total is $\mathcal{O}(n)$—**linear**, with no exponential 2^n term. To give a concrete magnitude: Roetteler–Naehrig–Svore–Lauter (2017) estimate the *entire* Shor-breaks-ECDLP circuit for secp256k1 ($n = 256$) at

$$9n + 2\lceil \log_2 n \rceil + 10 \ = \ 2330 \text{ logical qubits.}$$

The coefficient 9 is dominated by the *point accumulator* ($2n$) plus the *modular-inverse / field-arithmetic scratch* (several n), the latter the largest share; note that RNSV do *not* store two full $2n$-qubit input registers—they feed the scalar bits through a **semiclassical (measure-and-recycle) QFT**, so the scalar control costs only $\mathcal{O}(1)$ qubits, not the naive $2n$ of the count above (this attribution is illustrative—check the paper for the exact per-block breakdown before citing it).

That said, this challenge does *not* score this whole 2330; it scores only the peak qubits of the *single* point-add subcircuit—a subset of the machinery above. That per-primitive accounting (what the peak consists of, and why every qubit reclaimed in the modular inverse lowers the score) belongs to the implementation part—see §9.

5 Evolution of the Quantum State

This section writes out the steps of §3 as kets one by one, and *rigorously* derives the readout condition (4) previewed in step four—making clear "how the hidden period gets turned by interference into measurable peaks." For clarity, §5.3 first assumes the two input registers work over $\mathbb{Z}_r$ (ideal $M = r$); §5.4 then returns to the real $M = 2^t$ to discuss leakage.

5.1 Preparation and reversible evaluation

(1) Uniform superposition. Each input register is prepared into a uniform superposition over $\mathbb{Z}_r$ (on a real machine this is one layer of $H^{\otimes t}$, see §3):

$$|\psi_1\rangle = \frac{1}{r}\sum_{a=0}^{r-1}\sum_{b=0}^{r-1}|a\rangle\,|b\rangle\,|\mathcal{O}\rangle. \tag{6}$$

(2) Reversible evaluation of f. Apply U_f (Step 2), writing $aP + bQ$ into the point register (using $Q = kP$, so $aP + bQ = (a + kb)P$ is a multiple of P depending only on $a + kb \bmod r$; see (2)):

$$|\psi_2\rangle = \frac{1}{r}\sum_{a,b}|a\rangle\,|b\rangle\,\big|(a + kb \bmod r)\,P\big\rangle. \tag{7}$$

This step evaluates f over *all* r^2 pairs (a, b) at once (superposed evaluation). Note the point register's value depends only on $a+kb \bmod r$: along f's period direction $(-k, 1)$, the point register's value is **many-to-one**—all (a, b) connected by integer multiples of $(-k, 1)$ map to the *same* point value. This is exactly the quantum version of (3) (the point value is invariant along $(-k, 1)$), and the physical premise for interference to work.

5.2 Measure the value register: collapse to a coset (auxiliary viewpoint)

The most intuitive way to analyze the interference is to imagine (without actually doing it) *measuring the point register* now. Say we read the point s_0P, i.e. some $s_0 \in \mathbb{Z}_r$. By (7), the input registers then collapse to a uniform superposition over the coset satisfying $a + kb \equiv s_0$. That coset has exactly one $a = (s_0 - kb) \bmod r$ for each $b \in \mathbb{Z}_r$, r terms in all, so

$$|\psi_3\rangle = \frac{1}{\sqrt{r}}\sum_{b=0}^{r-1}\big|(s_0 - kb) \bmod r\big\rangle\,|b\rangle. \tag{8}$$

(As will be seen: *not* measuring the point register gives exactly the same result—it merely tags each coset with a label that can be separated from the input registers.)

5.3 Double QFT and the constructive-interference condition (ideal $M = r$)

Apply $\mathrm{QFT}_r : |x\rangle \mapsto \frac{1}{\sqrt{r}} \sum_y \omega_r^{xy} |y\rangle$ to each input register. Substituting (8):

$$\begin{aligned} |\psi_4\rangle &= \frac{1}{\sqrt{r}} \sum_b \left(\tfrac{1}{\sqrt{r}} \sum_c \omega_r^{(s_0 - kb)c} |c\rangle \right) \left(\tfrac{1}{\sqrt{r}} \sum_d \omega_r^{bd} |d\rangle \right) \\ &= \frac{1}{r^{3/2}} \sum_{c,d} \omega_r^{s_0 c} \underbrace{\left(\sum_{b=0}^{r-1} \omega_r^{b(d-kc)} \right)}_{(\star)} |c\rangle\, |d\rangle. \end{aligned} \tag{9}$$

The inner sum $(\star)$ is a complete geometric series, obeying the orthogonality relation

$$\sum_{b=0}^{r-1} \omega_r^{b(d-kc)} = \begin{cases} r, & d \equiv kc \pmod r, \\ 0, & \text{otherwise}, \end{cases} \tag{10}$$

i.e. **only when $d \equiv kc$ do the b-terms align in phase and add constructively; otherwise they spread uniformly around the unit circle and cancel completely**. So (9) simplifies to

$$|\psi_4\rangle = \frac{1}{\sqrt{r}} \sum_{c=0}^{r-1} \omega_r^{s_0 c} \, |c\rangle \, |kc \bmod r\rangle. \tag{11}$$

The measured (c, d) therefore **necessarily satisfies** $d \equiv kc \pmod r$—exactly the readout condition (4) previewed in step four of §2, now rigorously derived from interference. Each such c appears with equal probability $1/r$; the phase $\omega_r^{s_0 c}$ affects only the phase, not the probability, so the specific value of the imagined measured s_0 is *irrelevant*—this is exactly what "no need to actually measure the point register" means.

5.4 Back to $M = 2^t$: leakage, peak width, and rounding

A real machine uses $M = 2^t$, with b ranging over $\mathbb{Z}_M$, and the constraint $a \equiv s_0 - kb \pmod r$ has $\lfloor M/r \rfloor$ or $\lceil M/r \rceil$ solutions in $\mathbb{Z}_M$ for each b. Now the clean orthogonality of (10) no longer holds; the inner sum becomes a finite geometric series $\sum_m \omega_M^{rmc}$, whose modulus is a **Dirichlet kernel**:

$$\left| \sum_{m=0}^{L-1} \omega_M^{rmc} \right| = \left| \frac{\sin(\pi\, rcL/M)}{\sin(\pi\, rc/M)} \right|, \qquad L \approx M/r,$$

forming peaks of *width about* $1/M$ where rc/M is near an integer, and decaying rapidly elsewhere. The two-dimensional combined conclusion is: the measured (c, d) falls, with probability $\Omega(1)$, within the $1/M$ neighborhood of some integer point $(\hat{c}, \hat{d})$ satisfying $\hat{d} \equiv k\hat{c} \pmod r$, i.e.

$$\left| \tfrac{c}{M} - \tfrac{\hat{c}}{r} \right| \le \tfrac{1}{2M}, \qquad \left| \tfrac{d}{M} - \tfrac{\hat{d}}{r} \right| \le \tfrac{1}{2M}. \tag{12}$$

In other words, the only side effect of $M = 2^t$ is to *broaden* those clean integer points $(\hat{c}, \hat{d})$ of §5.3 into approximate rationals with $1/M$ jitter—requiring the next section's nearest-integer rounding to exactly recover $\hat{c}/r, \hat{d}/r$, resampling if necessary. (One could also take t large enough that $2^t \gtrsim r^2$, making $\hat{c}/r$ the *unique* rational with denominator $\le r$ inside the neighborhood of (12)—the continued-fraction condition of the *factoring* version. But that is **not needed here**: because r is public in ECDLP, $t \approx n$ already suffices, as §6 shows.)

6 Confirming the Period

This is the wrap-up "after the quantum measurement," and where the private key is finally determined—but it is a **sample → classical recovery → verify** closed loop, not a single step guaranteed to succeed at once. The emphasis on "confirming" is precisely because the quantum part only compresses the information about k into one candidate *with high probability*, and what truly *confirms* it is that final cheap classical check.

6.1 Recovering k from (c, d) (two-dimensional modular division)

Consider the ideal case first (§5.3): measurement directly gives (c, d) satisfying $d \equiv kc \pmod r$. As long as $c \not\equiv 0$, since r is prime and c is invertible mod r,

$$k \;\equiv\; d\,c^{-1} \pmod r. \tag{13}$$

One purely classical modular inverse yields the candidate k. The only bad sample is $c \equiv 0$ (probability $1/r$, negligible), where the formula is meaningless; discard and resample.

Rounding under real $M = 2^t$. Real measurement gives not the clean $(\hat{c}, \hat{d})$ but an approximation jittered by $1/M$ ((12)). The key point is that in ECDLP the order r is *public*: rounding $c\,r/M$ and $d\,r/M$ to the nearest integer recovers $\hat{c}$, $\hat{d}$, provided $M \gtrsim r$ (i.e. $t \approx n$, with rounding error $r/2M < 1/2$ guaranteeing uniqueness); substituting into (13) gives k. (Contrast: in the *factoring* version of Shor r is unknown, which is why one needs **continued fractions** to approximate a convergent with denominator $\le r$ from y/M, and to take $t \gtrsim 2n$; here r is known, $t \approx n$ suffices, and no continued fractions are needed.)

6.2 Verification, failure probability, and expected number of repetitions

Having obtained a candidate k, do one **cheap classical verification**: compute kP (one scalar multiplication, milliseconds classically), and check

$$kP \;\stackrel{?}{=}\; Q.$$

- Equal ⇒ the private key is recovered successfully, and the algorithm **terminates**. Because verification is deterministic, it never misjudges a wrong k as correct.
- Not equal ⇒ this sample landed on a bad point ($c \equiv 0$, or rounding recovery failed), so **go back to Step 1 of §3 and resample**, until verification passes.

The probability that one quantum run gives a usable sample is a constant $\Omega(1)$ (the fraction of good (c, d) times the probability $1 - 1/r$ that c is invertible), so **only $\mathcal{O}(1)$ resamples are expected**. This also shows that the algorithm's expected cost is still dominated by Step 2's $\Theta(n)$ reversible point additions—resampling merely multiplies it by a constant.

The loop thus closes: ECDLP $\xrightarrow{\text{encode}}$ the hidden period of f $\xrightarrow{\text{superposed eval+QFT}}$ samples satisfying $d \equiv kc$ $\xrightarrow{\text{rounding+mod division}}$ candidate k $\xrightarrow{kP \stackrel{?}{=} Q}$ confirmed private key. And the whole process's cost is almost entirely pressed onto Step 2, the **reversible point addition** called $\Theta(n)$ times—exactly what the ecdsa.fail challenge optimizes. As for why this "period" exists at all, and why f is chosen this way, the next section circles back to explain.

7 Why k Becomes a "Period"

The preceding sections gave the complete mechanical flow; this section circles back to explain the *intuition and design motivation* underpinning the whole construction—what is to be broken is $Q = kP$, and why k would manifest as a *period* extractable by the QFT. This section introduces no new formalism and only answers "why set the problem up this way."

How the period is "manufactured." This period is not something *intrinsically grown* out of $Q = kP$; rather, **we deliberately choose $f(a, b) = aP + bQ$ to manufacture it**. Look at when f repeats (takes the same point value):

$$f(a,b) = f(a',b') \iff (a-a')P = (b'-b)\,Q \overset{Q=kP}{\iff} a - a' \equiv k\,(b'-b) \pmod{r}.$$

Taking the simplest single step—incrementing b by 1 requires decrementing a by k to return to the same point:

$$f(a,b) = f(a-k,\ b+1).$$

Why exactly k? Because $Q = kP$: adding one more Q equals adding k more P's, so "b takes one step" is worth "a taking k steps"; to keep the value unchanged, back a off by k steps. This "one step of $b = k$ steps of a" holds everywhere and repeats regularly—**the period arises thereby, and its ratio is exactly k.**

Aside: why this attack uses $f = aP + bQ$. The point addition this challenge optimizes is precisely the inner building block computing this probe function $f(a, b) = aP + bQ$ (§4). Taking f as $aP + bQ$ is not arbitrary; there are several considerations behind it, which also explain why it looks the way it does:

1. **It is linear (a group homomorphism).** f maps $\mathbb{Z} \times \mathbb{Z}$ homomorphically into the curve group, $(1, 0) \mapsto P$, $(0, 1) \mapsto Q$. Linearity makes "the (a, b) that leave f invariant" form exactly a *subgroup / lattice*, rather than a messy set—so the period is a regular lattice, and the QFT (essentially a Fourier transform on the group) can read it out easily; a nonlinear f generally does not give such a clean period.

2. **It uses both P and Q at once.** k hides in the *coupling* of the two: looking at a single axis, the periods of $f(a, 0) = aP$ and $f(0, b) = bQ$ are both the *public* r; it is "adding aP and bQ," borrowing $Q = kP$, that ties the two axes together at ratio k, so that k shows up as a slope. This is also why it has one more dimension than the factoring version.

3. **It can be computed efficiently and reversibly.** $aP + bQ$ can be realized reversibly in poly(n) gates via double-and-add (U_f)—this is the practical threshold for whether it can actually run on a quantum machine.

4. **It satisfies the "hiding" condition.** $f(a, b) = (a + kb \bmod r)P$ takes the same value on the same line $a + kb \equiv$constant and *different* values on different lines (corresponding to r distinct points), so the period is "clean" and can be read out unambiguously—exactly the hiding property the abelian hidden subgroup problem requires.

The natural landing point satisfying these is $f = aP + bQ$.

Part II
Implementation: What the ecdsafail-challenge Actually Does

Part I covered the *general* algorithmic principle; this part maps it onto the actual secp256k1 point-add challenge source: which segment the challenge actually implements, and which file / function each section of Part I lands in. (Paths below are relative to the ecdsafail-challenge repo root; line numbers drift with refactoring, so only file and function names are given.)

8 Which segment this challenge actually implements

The challenge implements only the **single point addition** primitive: given a quantum point target (x, y) (qubits) and a classical point offset (x, y) (classical bits), reversibly overwrite target *in place* with the affine sum target + offset (on secp256k1). It **does not include** the QFT, measurement, the whole $aP + bQ$, and has no a, b input registers—these are the *algorithm layer* of Part I ("why it is done this way"), which have no code entity here; what the challenge corresponds to is only the innermost *single point-add step* (Part I's Step 2, §4).

Why single out just this segment. The most direct reason is that it can be *classically* verified: a single point addition is a reversible *classical permutation*, checkable point by point on an ordinary machine against 9024 test points; whereas the full Shor with QFT / true superposition either cannot be classically simulated or requires a real quantum machine (which would already amount to a real break)—point addition is the largest chunk that "can still be checked on a laptop." As for the other reasons—it is the *bottleneck* of the whole attack, amplified $\approx 2n$ times as the innermost loop, and the only component still with algorithmic freedom left—those are exactly the themes to be unpacked in §9 and §10.

Interface and scoring (the contract). The circuit must consume four 256-element registers—`target_x`, `target_y` (qubits) and `offset_x`, `offset_y` (classical bits)—and overwrite `target_x`, `target_y` *in place* with the affine sum. The scoring is consistent with Part I's "why press Toffoli×qubit": score = $\overline{\text{Toffoli}}\times$peak qubits, *lower is better* (the industrial meaning of this metric is in §9).

What counts as "valid": the 0/0/0 conditions. A submission must pass on all 9024 test points; failing any single one means a loss. The name "0/0/0" refers to the three error channels—classical, phase, and ancilla—all reading zero; concretely this unpacks into the following checks:

1. **Classically correct.** All 9024 points must compute the correct (R_x, R_y).
2. **Reversible (ancilla zeroed).** Every ancilla must be uncomputed back to $|0\rangle$ before release; after the forward pass, all non-output qubits must also return to $|0\rangle$.
3. **Phase-clean.** The global phase of all surviving branches is zero—no phase kickback left by incomplete uncomputation.
4. **Forward∘inverse identity.** Running the circuit once and then its gate-level inverse must restore the initial state qubit by qubit.

These four conditions block all "fake wins": any drop in Toffoli count bought by skipping uncomputation, leaking phase, or dumping garbage into ancilla only makes this submission *lose* rather than run faster. The reason "reversible + phase-clean" is enforced even for a primitive verified only on *classical* points is that it will eventually be embedded in a real Shor, acting on a *superposition* (§4): then unzeroed ancilla and residual phase would pollute the interference and destroy the whole attack—these two conditions are exactly the threshold that upgrades "classically correct" to "quantum-usable."

What can and cannot be changed, and why.

- **Changeable:** everything in `src/point_add/`—split modules, rewrite the primitive, swap algorithms, refactor freely. All the optimization space is here.
- **The unchangeable "contract layer":** the harness—`src/lib.rs`, `circuit.rs`, `sim.rs`, `weierstrass_elliptic_curve.rs`, and the `src/bin/` drivers—which defines the interface, the counting rules (counting every Toffoli / Clifford / peak qubit), the 0/0/0 checks, and the scoring; plus `Cargo.toml` / `Cargo.lock` / `rust-toolchain` (no new dependencies allowed), and no writing directly to `results.tsv`.

The design intent is plain: *separate the referee from the player.* If the counting, checking, and reference curve could be changed, one could cheat by changing "how it counts," and the score would lose meaning; locking down the harness guarantees the score reflects only *the algorithm itself* and that different submissions are comparable. The test points are derived by **Fiat–Shamir** hashing of the submitted op stream, so players cannot tune to the test set (cannot "memorize" the circuit to work only on fixed point pairs); the no-new-dependencies rule works the same way, preventing cost from being outsourced to a library. So the only way to lower the score is to genuinely make that modular inverse of §10 cheaper—which is exactly what the competition is designed to force out.

9 Why industry wants to minimize Toffoli count × qubit count

§8 already gave the scoring scalar score = $\overline{\text{Toffoli}} \times$peak qubits (lower is better); this section clarifies its *industrial motivation*—why the metric of greatest concern is exactly the **product of executed Toffoli gates and occupied qubits**, and not the total gate count or anything else.

First, pin down which step this score measures. Back to §3 and §4.4: across the whole algorithm, preparing the superposition (one layer of H) and the two QFTs are only cheap single/two-qubit gates, so **almost all Toffolis come from Step 2's reversible evaluation** U_f, and the circuit's peak qubits are also reached during U_f (those wide modular-inverse scratch registers). So the two quantities in the score—executed Toffoli count and peak qubits—**essentially measure Step 2**, more precisely the execution cost of its innermost **point-add primitive**, called repeatedly by double-and-add. In a word: *scoring the resources of the whole Shor attack is roughly scoring this one point-add primitive*—Steps 1 and 3 barely count.

What the scored peak actually is (and how big). Concretely, the scored peak is that single point-add subcircuit's own footprint: *point accumulator* ($\approx 2n$) + *modular-inverse scratch* (several n), reaching its high-water mark during the modular-inverse phase (§4.3), and **excluding** the two $2n$ input registers (they belong to the *outer* double-and-add loop, and do not enter a *single* point

addition). So it is a strict *subset* of the machinery counted in the 2330 of §4.5—magnitude $\sim 4.5n$ (currently measured at a frontier peak of ≈ 1150 qubits), *not* "2330 minus the input registers." Because the score is $\overline{\text{Toffoli}} \times$peak qubits, **every qubit reclaimed in the modular inverse directly lowers the score**, exactly as it lowers the Toffoli factor—which is why §10 treats the modular inverse as the joint cost center of both factors.

So why measure it as "the two multiplied together"? Because this product is a clean proxy for the **space–time volume of fault-tolerant quantum computing (FTQC)**, and space–time volume is the real physical cost:

- **Toffoli / T gates are the only expensive gates (the time axis).** Under mainstream error-correcting architectures like the surface code, Clifford gates are nearly free, whereas every Toffoli (or T gate) must be supplied by *magic state distillation*—distillation factories occupy the vast majority of a fault-tolerant machine's physical cost and runtime. So "average executed Toffoli count" is directly proportional to the distillation load, roughly the *runtime* of this computation.
- **Peak qubit count determines how big the machine must be (the space axis).** Each logical qubit costs thousands of physical qubits under error correction; the peak width determines "how wide the machine must at least be," roughly the machine's *scale*.
- **The product of the two = space–time volume.** Multiplying "runtime" by "machine scale" gives the space–time resources this computation occupies on a fault-tolerant machine—the real cost and engineering effort.

Now layer on the amplification effect stressed repeatedly here: point addition is called $\Theta(n)$ ($n = 256$) times in one complete Shor attack (the double-and-add of (5)), so **any constant-factor reduction on a single point addition multiplies by several hundred along the whole algorithm**. The number industry really cares about—"how big a fault-tolerant machine and how many hours it takes to break secp256k1 with a quantum computer"—is proportional to this amplified space–time volume; pressing down the point addition's Toffoli × qubit is equivalent to *directly revising down* this threat estimate. And this estimate is precisely the quantitative basis for the NIST post-quantum standardization process and for the ECDSA-deprecation timelines in finance / blockchain / TLS and other industries.

And within this space–time volume to be minimized, *which part* has the most optimization room? The answer is highly concentrated—the next section explains.

10 Why the main battlefield of optimization is the GCD (modular inverse)

The previous section explained *what to press*—the space–time volume Toffoli × qubit; this section explains *where to press*. The answer is highly concentrated: the **modular inverse** inside point addition, i.e. the step computed by binary GCD / Kaliski (§4.3). One might say **much of the optimization history of this circuit has been the optimization history of the modular inverse (GCD)**—though, as noted below, the surrounding field arithmetic still yields wins too. There are four reasons the GCD dominates, and they compound:

(1) It is the only iterative big-cost component in point addition. In one point addition ((1)), addition / subtraction / multiplication / squaring are all *branch-free, single-pass* circuits with fixed structure, and the adder (Gidney-style) and schoolbook / Karatsuba multiplication are already heavily optimized (though *not* provably minimal—the field arithmetic still yields the occasional win, see the end of the section). Only the modular inverse has *no* single-pass algorithm; it can only iterate GCD for $\Theta(n)$ (about 256–512) rounds, each round a compare + conditional subtract + shift. One multiplication is one pass, GCD is n passes—so **the Toffolis of a single point addition come mainly from the GCD loop body**.

(2) It maxes out both factors of the score at once. The score is a *product* Toffoli × qubit, so the component that can lower both factors yields the most, and the GCD occupies both:

- **Toffoli axis**: $\Theta(n)$ rounds × the comparator / conditional subtraction per round, the bulk of the point addition's gate count.
- **Qubit axis**: Kaliski must maintain several n-bit state registers (u, v, r, s and the like) plus scratch at once, and **the peak width of the whole circuit is often reached during the GCD phase**.

Other components usually improve only one factor; the GCD is one of the few that *affects both factors at once*, with the largest marginal gain on the product.

(3) Reversibility doubles it, double-and-add amplifies it. The scratch produced by the modular inverse must be uncomputed to zero (the second Kaliski pass, §4.3), so saving one share on the GCD actually saves two; and the modular inverse is called once per point addition, and point addition is called $\Theta(n)$ times by double-and-add ((5)), so any constant factor pressed down here **multiplies by several hundred along the whole algorithm**. Savings on other components are additive; savings on the GCD are multiplicative.

(4) It has the most algorithmic freedom left. The adder / multiplication / squaring are structurally fixed single-pass circuits, already close to their practical limits—so the place in one point addition where the *biggest* trade-offs remain—*which modular-inverse algorithm to choose and how to organize its iterations*—is concentrated in the GCD, which is still an open design problem. This does not mean the field arithmetic is frozen: it is heavily optimized but not provably minimal, and recent frontier moves have still come from restructuring it—for instance, rewriting the modular *squaring* from an AND-based schoolbook form into a controlled add–subtract (roughly halving the Toffolis per cross product) delivered a few-percent score improvement without touching the GCD. But such wins are the exception; the largest and most reliable leverage is still the modular inverse, and focusing on it is not a preference but what the circuit structure dictates.

In a word: minimizing Toffoli × qubit is, to first order, reducing the cost of the modular inverse—the GCD is the **largest cost center, the amplifier, and the richest locus of algorithmic freedom** of a single point addition, with the surrounding field arithmetic a secondary lever. This is also why, on this circuit, most advances that push cost to a new low ultimately trace back to the same place: lowering the cost of that one modular-inverse step.

11 Where the code is: peeling inward along point addition

Peeling inward along "a single point addition," the code has roughly four layers, each more of an optimization focus the deeper it goes:

- **Top-level point addition** — `ec_add()` in `src/point_add/trailmix_ludicrous/ec_add.rs` (entry `point_add/mod.rs::build()`). It adds a classical point in place to the quantum point, using exactly Part I's addition law (1) for λ, x_3, y_3 (the file header comment is "a-independent secp add formula," just optimized in place).

- **The most expensive part—the modular inverse** — `gcd.rs`, the jump-GCD for $y \mapsto y \cdot x^{-1} \bmod q$ (here $q = p$ is the field prime—the code's variable name for it; not the group order r). This is the entity referred to by §4.3's "most expensive, computed twice" and §10's "the GCD is the cost center," and the file actually optimized over and over; the schedule / comparator / venting / constant-propagation machinery around it is in `schedule.rs`, `comparator.rs`, `venting.rs`, and `constprop.rs`.

- **The rest of the arithmetic** (modular multiplication / squaring / addition / comparison — heavily optimized, though still an occasional source of wins, e.g. a recent restructuring of the modular squaring) — `src/point_add/arith/` and `gidney.rs`, `multiply.rs`, `square.rs`.

- **Verification and scoring** — `src/sim.rs` runs 0/0/0 on the 9024 test points and counts Toffolis and peak qubits; `src/bin/eval_circuit.rs` sets the secp256k1 parameters, computes the reference point addition, and scores $\overline{\text{Toffoli}} \times$peak qubits (§9). The curve's classical reference is in `src/weierstrass_elliptic_curve.rs`.

In a word: the whole codebase is the implementation of Part I's innermost "single point-add step," and the deeper in (the modular inverse / GCD), the more it is the optimization battlefield.